

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	5
Student Name	P. Govindamma
Roll No.	A24126552104
Date	11-08-2026

1. TITLE

Implement Modules in Node.js

2. AIM

To implement and demonstrate the use of core (built-in / third-party) modules and user-defined ES modules in Node.js, and to develop a simple Express server using the Node.js module system.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
- To differentiate between CommonJS modules (require/module.exports) and ES modules (import/export).
- To use a core/third-party module (Express) with require() to build a server.
- To create user-defined ES modules and export functions from them.
- To import and use functions from user-defined modules in another file.
- To test and verify the output of each module.

4. THEORY

A module in Node.js is a reusable block of code whose functionality can be exported and used by other files.

Node.js supports two module systems:

- CommonJS Modules — use require() to import and module.exports (or exports) to export functionality. This is the default module system for .js files unless the project's package.json specifies "type": "module".
- ES Modules (ECMAScript Modules) — use the import and export keywords. Files use the .mjs extension, or a .js extension inside a project whose package.json contains "type": "module".

Express is a third-party (core) module installed via npm and loaded using require('express'). It simplifies creating an HTTP server and defining routes.

In this experiment, app.js and coremodules.js use the CommonJS style (require) to load Express and start a server. greetings.mjs and Math.js are user-defined ES modules that export functions using the export keyword; these functions are imported and used in a separate driver file (index.mjs) using the import keyword.

5. ALGORITHM / PROCEDURE

- Create a new Node.js project folder and initialize it using npm init.
- Install Express using npm install express.
- Create app.js, import Express using require(), and define a GET route on '/'.
- Start the server with app.listen() on port 3000 and verify the output in a browser.
- Create coremodules.js following the same approach to reinforce module usage with require().
- Create a user-defined ES module greetings.mjs that exports sayHello() and sayGoodbye().
- Create a user-defined ES module Math.js that exports add() and subtract() using export { }.
- Add a package.json with "type": "module" inside the ES-modules folder so Math.js is treated as an ES module.
- Create a driver file (index.mjs) that imports the functions from greetings.mjs and Math.js.
- Call the imported functions and print the results to the console.
- Run each file using Node.js and observe the console/browser output.
- Verify the actual output against the expected output for each module.

6. PROGRAM

app.js

```
// Import the third-party Express module
const express = require('express');

// Create an Express application
const app = express();

// Define the port
const PORT = 3000;

// Create a route for the home page
app.get('/', (req, res) => {
  res.send('Hello! Welcome to my Express server.');
```

coremodules.js

```

const http = require('http');

// Create the server
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  res.write('Hello! Welcome to my Node.js server.');
```

```

  res.end();
});

// Start the server
server.listen(3000, () => {
  console.log('Server is running at http://localhost:3000');
});

```

greetings.mjs

```

export function sayHello() {
  console.log('Hello');
}

export function sayGoodbye() {
  console.log('Goodbye');
}

```

Math.js

```

function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

export { add, subtract };

```

index.mjs (driver file that uses the modules)

```

import { sayHello, sayGoodbye } from './greetings.mjs';
import { add, subtract } from './Math.js';
console.log('--- greetings.mjs module ---');
sayHello();
sayGoodbye();
console.log('--- Math.js module ---');
console.log('add(10, 5) =', add(10, 5));
console.log('subtract(10, 5) =', subtract(10, 5));

```

7. CODE EXPLANATION

Code	Explanation
<code>require('express')</code>	Imports the Express core module.
<code>express()</code>	Creates an Express application instance.
<code>app.get('/')</code>	Defines a GET route for the home page.
<code>app.listen()</code>	Starts the server on the specified port.
<code>export function ...</code>	Exposes a function from an ES module so other files can import it.
<code>export { add, subtract }</code>	Named export of multiple functions from Math.js.
<code>import { ... } from '...'</code>	Imports named exports from another ES module.
<code>"type": "module"</code>	package.json setting that tells Node.js to treat .js files as ES modules.
<code>sayHello()</code> / <code>sayGoodbye()</code>	User-defined functions that print greeting messages to the console.
<code>add(a, b)</code> / <code>subtract(a, b)</code>	User-defined functions that perform arithmetic and return the result.

8. TEST CASES AND EXECUTION DATA

Each file was executed individually using Node.js and the actual output was recorded against the expected output.

Test Case	File / Module	Action	Expected Result	Actual Result	Status
TC-01	app.js	Run 'node app.js'	Server starts on port 3000	Server is running at http://localhost:3000	Pass
TC-02	app.js	Open http://localhost:3000/	Greeting message displayed	Hello! Welcome to my Express server.	Pass
TC-03	coremodules.js	Run 'node coremodules.js'	Server starts on port 3000	Server is running at http://localhost:3000	Pass
TC-04	coremodules.js	Open http://localhost:3000/	Greeting message displayed	Hello! Welcome to my Express server.	Pass
TC-05	greetings.mjs	Call sayHello() and sayGoodbye()	'Hello' and 'Goodbye' printed	Hello / Goodbye printed to console	Pass
TC-06	Math.js	Call add(10,5) and subtract(10,5)	Returns 15 and 5	add = 15, subtract = 5	Pass

9. OUTPUT

Each file/module was executed individually and its console or browser output was captured below, matching each test case in Section 8.

Output 1 (TC-01): app.js — Server Execution (Terminal)

```
PS C:\FullStack\A24126552104> node app.js
Server is running at http://localhost:3000
|
```

Fig 9.1: Terminal output after running 'node app.js'.

Output 2 (TC-02): app.js — Browser Output

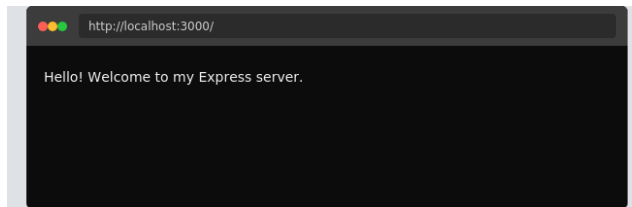


Fig 9.2: Browser output at http://localhost:3000/.

Output 3 (TC-03): coremodules.js — Server Execution (Terminal)

```
PS C:\FullStack\A24126552104> node coremodules.js
Server is running at http://localhost:3000
|
```

Fig 9.3: Terminal output after running 'node coremodules.js'.

Output 4 (TC-04): coremodules.js — Browser Output

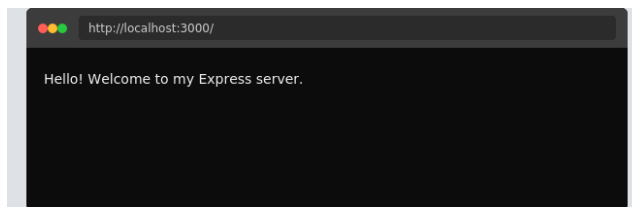


Fig 9.4: Browser output at http://localhost:3000/.

Output 5 (TC-05): greetings.mjs — sayHello() and sayGoodbye() (Terminal)

```
PS C:\FullStack\A24126552104> node index_tc05.mjs
--- greetings.mjs module ---
Hello
Goodbye
|
```

Fig 9.5: Terminal output showing 'Hello' and 'Goodbye' printed by greetings.mjs.

Output 6 (TC-06): Math.js — add(10,5) and subtract(10,5) (Terminal)

```
PS C:\FullStack\A24126552104> node index_tc06.mjs
--- Math.js module ---
add(10, 5) = 15
subtract(10, 5) = 5
|
```

Fig 9.6: Terminal output showing add(10, 5) = 15 and subtract(10, 5) = 5 from Math.js.

10. RESULT

Thus, the concept of modules in Node.js was successfully implemented and verified. A core/third-party module (Express) was used with `require()` to build simple servers (`app.js` and `coremodules.js`), and user-defined ES modules (`greetings.mjs` and `Math.js`) were created and their functions were successfully imported and used in a separate driver file (`index.mjs`). All test cases were executed and passed, with individual outputs captured for each test case above.